

TMDB Movie Recommendation System



Prepared by: Divya Upadhyay
02/10/2026
GitHub Link: [\[GitHub\]](#)

Outline

- Executive
- Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary (Methodologies)

- The project follows a **content-based recommendation approach** using the **TMDB 5000 dataset**. The methodology includes **data preparation, weighted rating analysis, popularity analysis, feature scaling, hybrid ranking, NLP-based TF-IDF vectorization, similarity calculation, and recommendation ranking**. The system uses movie overviews to identify and recommend movies with similar content.
- **Data Preparation:** Merged and cleaned the TMDB 5000 Movies and Credits datasets.
- **Rating Analysis:** Used weighted ratings based on vote average and vote count.
- **Popularity Analysis:** Ranked movies using popularity scores.
- **Hybrid Ranking:** Combined scaled weighted ratings and popularity.
- **NLP:** Applied TF-IDF to movie overview text.
- **Similarity:** Used sigmoid-kernel similarity to identify similar movies.
- **Recommendation:** Generated a ranked list of the top 10 content-similar movies.
- **GitHub Link:** [\[GitHub\]](#)

Executive Summary (Results)

- **Weighted Rating:** Identified highly rated movies while considering the number of votes.
- **Popularity Analysis:** Ranked movies based on their popularity scores.
- **Hybrid Ranking:** Combined weighted rating and popularity to create a balanced ranking.
- **Content Similarity:** Used TF-IDF representations of movie overviews to measure content similarity.
- **Recommendations:** Generated a **Top 10 list of movies similar to a selected movie** using similarity scores.
- **Final Outcome:** Built a **content-based movie recommendation system** that recommends movies based on their textual content
- **GitHub Link:** [\[GitHub\]](#)

Introduction

- **Movie recommendation systems** help users discover relevant movies from large collections of content.
- This project focuses on **content-based movie recommendation**, where recommendations are generated based on movie content.
- The **TMDB 5000 dataset** is used to analyze movie information such as ratings, popularity, and descriptions.
- **Natural Language Processing (NLP)** is applied to movie overviews using **TF-IDF**.
- A **similarity-based approach** is then used to identify movies with similar content.
- The final system generates a **Top 10 list of similar movies** for a selected movie.
- **GitHub Link:** [[GitHub](#)]

Data Collection & Project Objective

- **Data Collection**
- Used the **TMDB 5000 Movies dataset**.
- Two datasets were used:
 - tmdb_5000_movies.csv
 - tmdb_5000_credits.csv
- Both datasets were merged using the common **movie ID**.
- The final dataset contains approximately **4,803 movies**.
- **GitHub Link:** [\[GitHub\]](#)

```
credits = pd.read_csv("tmdb_5000_credits.csv")
credits.head(4)
```

[21]

...

	movie_id	title	cast	crew
0	19995	Avatar	[{"cast_id": 242, "character": "Jake Sully", "...	[{"credit_id": "52fe48009251416c750aca23", "de...
1	285	Pirates of the Caribbean: At World's End	[{"cast_id": 4, "character": "Captain Jack Spa...	[{"credit_id": "52fe4232c3a36847f800b579", "de...
2	206647	Spectre	[{"cast_id": 1, "character": "James Bond", "cr...	[{"credit_id": "54805967c3a36829b5002c41", "de...
3	49026	The Dark Knight Rises	[{"cast_id": 2, "character": "Bruce Wayne / Ba...	[{"credit_id": "52fe4781c3a36847f81398c3", "de...

```
movies = pd.read_csv("tmdb_5000_movies.csv")
movies.head(4)
```

[22]

Data Preparation

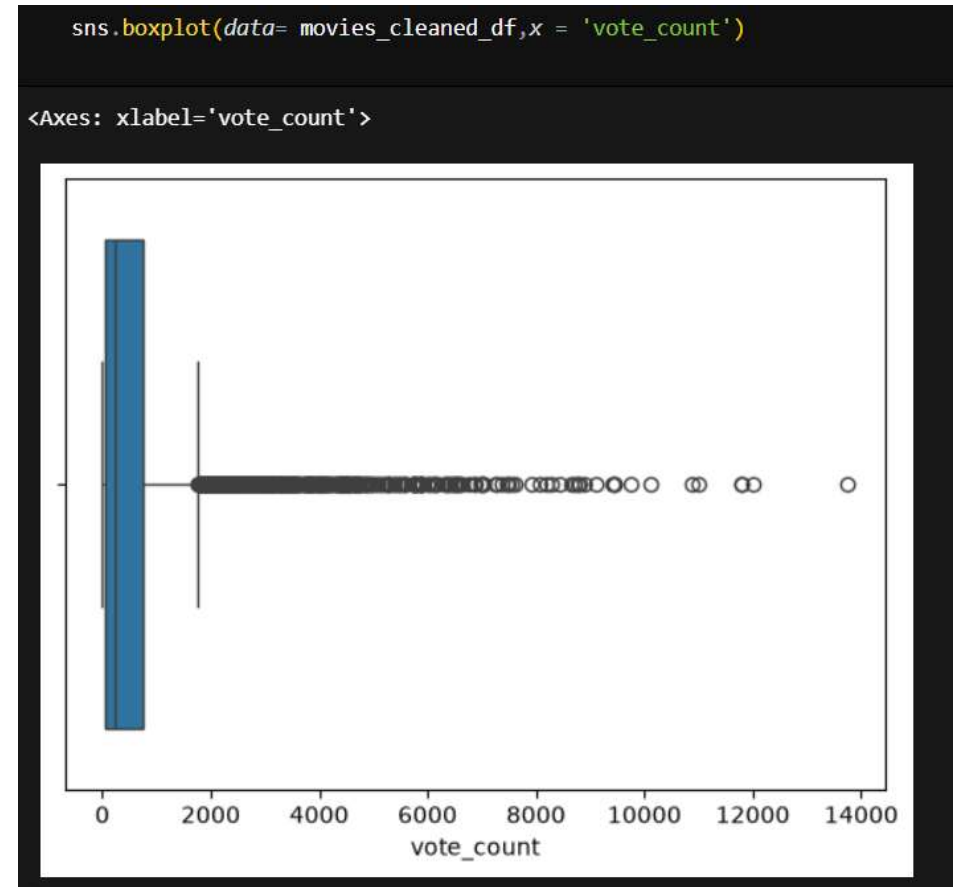
- Renamed movie_id to id for dataset alignment.
- Merged Movies and Credits datasets using id.
- Removed unnecessary columns:
 - homepage
 - title_x
 - title_y
 - status
 - production_countries
- Prepared the cleaned dataset for **rating analysis, popularity analysis and recommendation**.
- **Key Features**
- title • overview • vote_average • vote_count • popularity
- **GitHub Link:** [\[GitHub\]](#)

```
movies_cleaned_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4803 entries, 0 to 4802
Data columns (total 18 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   budget              4803 non-null  int64  
1   genres              4803 non-null  str     
2   id                  4803 non-null  int64  
3   keywords            4803 non-null  str     
4   original_language   4803 non-null  str     
5   original_title       4803 non-null  str     
6   overview            4800 non-null  str     
7   popularity          4803 non-null  float64 
8   production_companies 4803 non-null  str     
9   release_date        4802 non-null  str     
10  revenue             4803 non-null  int64  
11  runtime             4801 non-null  float64 
12  spoken_languages    4803 non-null  str     
13  tagline             3959 non-null  str     
14  vote_average        4803 non-null  float64 
15  vote_count          4803 non-null  int64  
16  cast                4803 non-null  str     
17  crew                4803 non-null  str     
dtypes: float64(3), int64(4), str(11)
memory usage: 38.4 MB
```

Rating Analysis & Weighted Rating

- **Movie Rating Analysis**
- Analyzed `vote_average` and `vote_count`.
- Movies with very few votes may have unreliable ratings.
- Used the **95th percentile of vote count** to determine the minimum vote threshold `m`.
- Filtered movies that did not meet this threshold.
- **Weighted Rating**
- Combined **rating quality + number of votes** to produce a more reliable movie ranking.
- **Result:**
Movies were ranked based on a more balanced rating rather than average rating alone.
- **GitHub Link:** [\[GitHub\]](#)

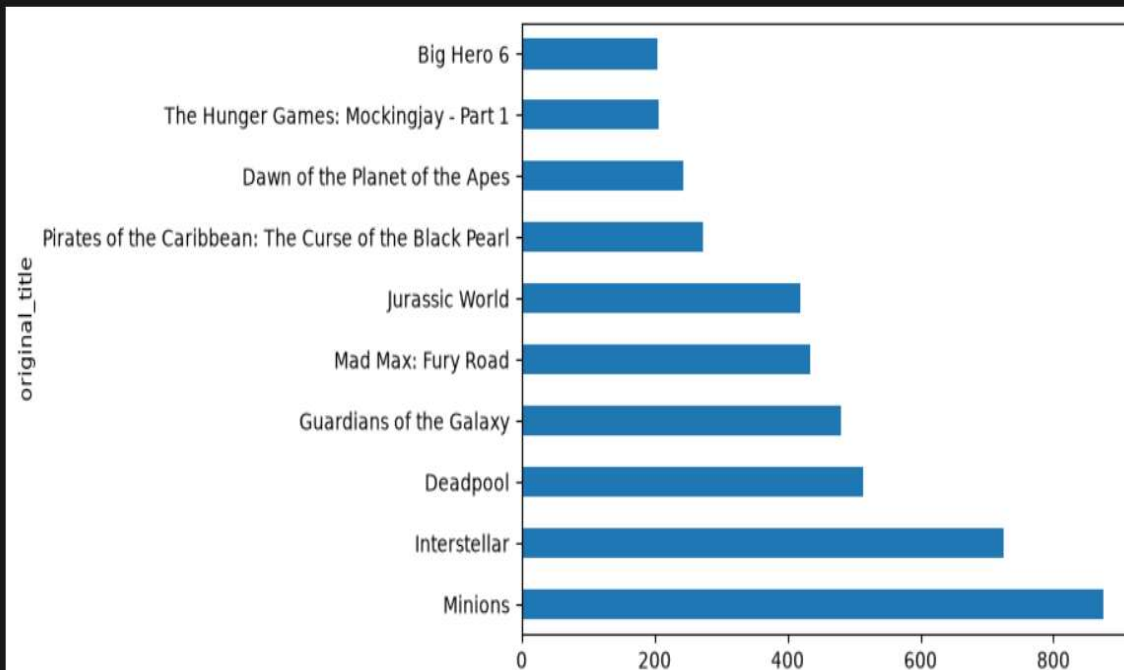


Popularity & Hybrid Ranking

- **Popularity Analysis**
- Ranked movies using their **popularity score**.
- Identified the most popular movies in the dataset.
- **Hybrid Ranking**
- Combined:
- **Weighted Rating + Popularity**
- Applied MinMaxScaler to scale both features.
- Used equal weights:
 - 50% Weighted Rating
 - 50% Popularity
- **Purpose**
- Create a balanced ranking considering both **movie quality and popularity**.
- GitHub Link: [\[GitHub\]](#)

```
popularity_df.set_index('original_title')['popularity'][:10].plot(kind='barh')
```

<Axes: ylabel='original_title'>



NLP & Movie Overview Processing

- The movie **overview** column was used as the main textual feature.
- **Why NLP?**
- Movie overviews contain information about:
 - Story
 - Characters
 - Themes
 - Events
 - Movie concepts
- NLP converts this textual information into a form that can be mathematically compared.
- **Goal**
- Transform movie descriptions into **numerical representations** for similarity calculation.
- GitHub Link: [\[GitHub\]](#)

```
df['overview'][3]
```

```
"Following the death of District Attorney Harvey Dent, Batman assumes responsibility for Dent's
```

```
tfidf_matrix.toarray()[0]
```

```
array([0., 0., 0., 0., 0.,  
       0., 0., 0., 0., 0.,  
       0., 0., 0., 0., 0.,  
       0., 0.33390581, 0., 0., 0.,
```

TF-IDF Vectorization

Applied TF-IDF (Term Frequency–Inverse Document Frequency) to movie overviews.

Configuration

- `ngram_range = (1,3)`
 - Unigrams
 - Bigrams
 - Trigrams
- `stop_words = "english"`
- `min_df = 3`

What happens?

Movie Overview → TF-IDF Vector → Numerical Representation

Each movie is represented as a numerical vector based on the importance of words in its overview.

- [GitHub Link: \[GitHub\]](#)

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(min_df = 3,max_features=None,stop_words='english',ngram_range=(1,3))

tfidf

> TfidfVectorizer ⓘ ⓘ

tfidf_matrix = tfidf.fit_transform(df['overview'])

tfidf_matrix

<Compressed Sparse Row sparse matrix of dtype 'float64'
with 3267 stored elements and shape (241, 610)>
```

Similarity & Recommendation Engine

- **Similarity Calculation**
- Applied the **sigmoid kernel** to the TF-IDF representations.
- Generated similarity scores between movies.
- Higher similarity → more similar movie content.
- **Recommendation Process**
- **Selected Movie**
 - ↓
 - Movie Overview**
 - ↓
 - TF-IDF Representation**
 - ↓
 - Similarity Scores**
 - ↓
 - Sort by Similarity**
 - ↓
- **Top 10 Similar Movies**
- The selected movie itself is excluded from the final recommendations.
- **GitHub Link:** [\[GitHub\]](#)

```
from sklearn.metrics.pairwise import sigmoid_kernel

sig = sigmoid_kernel(tfidf_matrix, tfidf_matrix)

sig

array([[0.76228178, 0.76159416, 0.76159416, ..., 0.76159416, 0.76159416,
        0.76159416],
       [0.76159416, 0.76228178, 0.76159416, ..., 0.76159416, 0.76159416,
        0.76159416],
       [0.76159416, 0.76159416, 0.76228178, ..., 0.76159416, 0.76159416,
        0.76159416],
       ...,
       [0.76159416, 0.76159416, 0.76159416, ..., 0.76228178, 0.76159416,
        0.76159416],
       [0.76159416, 0.76159416, 0.76159416, ..., 0.76159416, 0.76228178,
        0.76159416],
       [0.76159416, 0.76159416, 0.76159416, ..., 0.76159416, 0.76159416,
        0.76228178]], shape=(241, 241))

pd.DataFrame(sig)
```

Final System & Results

- The project combines:
- **Data Collection → Data Cleaning → Rating Analysis → Popularity Analysis → Hybrid Ranking → NLP → TF-IDF → Similarity → Recommendation**
- **Results**
- Generated reliable **weighted movie rankings**.
- Identified movies based on **popularity**.
- Created a balanced **hybrid ranking**.
- Converted movie overviews into numerical features using **TF-IDF**.
- Generated **Top 10 content-similar movie recommendations**.
- **Final Outcome**
- A **content-based movie recommendation system** that recommends movies based on the **textual similarity of their movie overviews**.
- **GitHub Link:** [\[GitHub\]](#)

```
from sklearn.metrics.pairwise import sigmoid_kernel

sig = sigmoid_kernel(tfidf_matrix,tfidf_matrix)

sig

array([[0.76228178, 0.76159416, 0.76159416, ..., 0.76159416, 0.76159416,
        0.76159416],
       [0.76159416, 0.76228178, 0.76159416, ..., 0.76159416, 0.76159416,
        0.76159416],
       [0.76159416, 0.76159416, 0.76228178, ..., 0.76159416, 0.76159416,
        0.76159416],
       ...,
       [0.76159416, 0.76159416, 0.76159416, ..., 0.76228178, 0.76159416,
        0.76159416],
       [0.76159416, 0.76159416, 0.76159416, ..., 0.76159416, 0.76228178,
        0.76159416],
       [0.76159416, 0.76159416, 0.76159416, ..., 0.76159416, 0.76159416,
        0.76228178]], shape=(241, 241))

pd.DataFrame(sig)
```

Thankyou!